

REAL TIME PARKING

โดย

นาย สุภณัฐ กุ่มครอง 6607174

นาย กฤตภาส มีมาก 6607612

นาย วิชัยศ เงินศิริ 6608624



วัตถุประสงค์โครงการ

1.

เพื่อนำไปใช้
จัดการระบบ
ในที่จอดรถ

2.

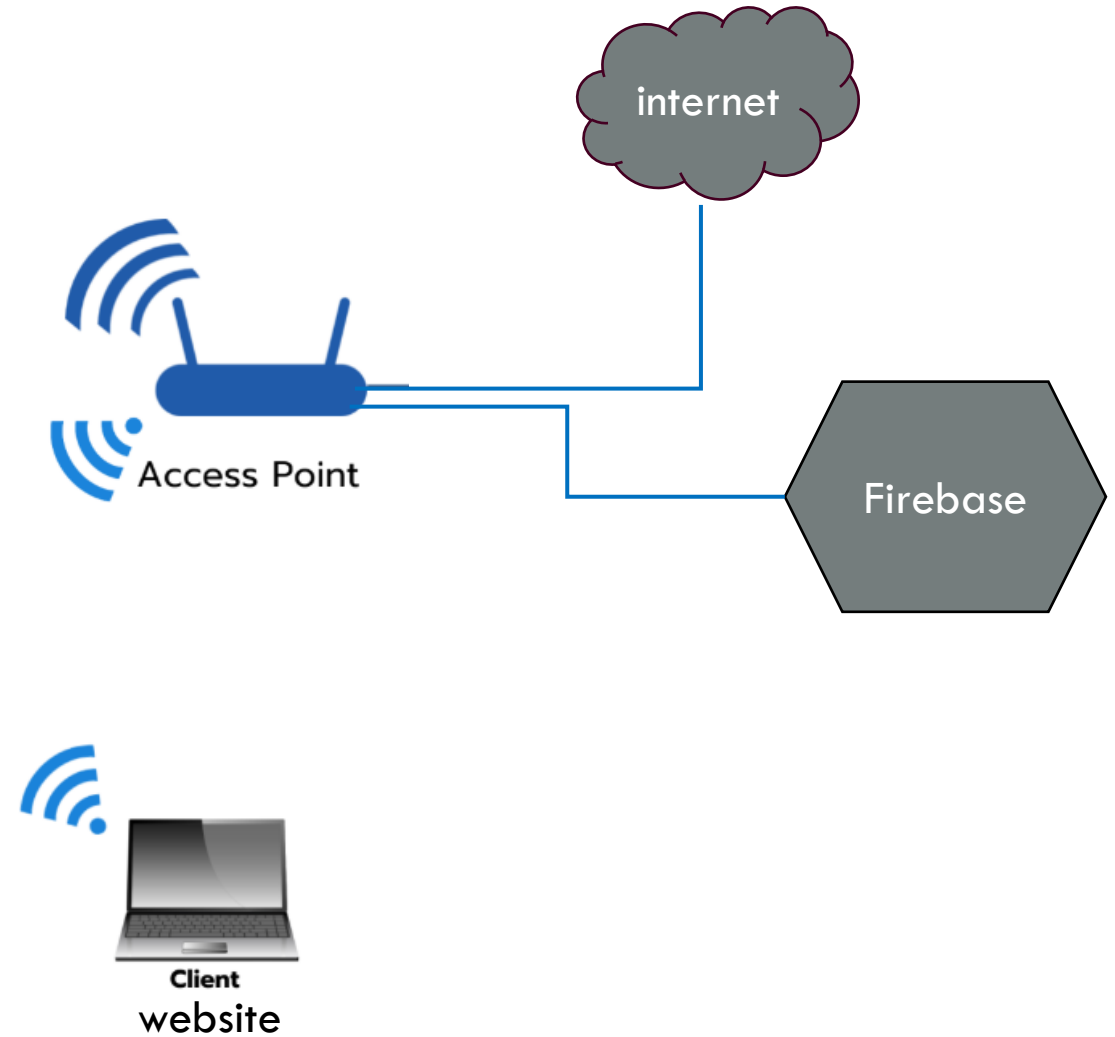
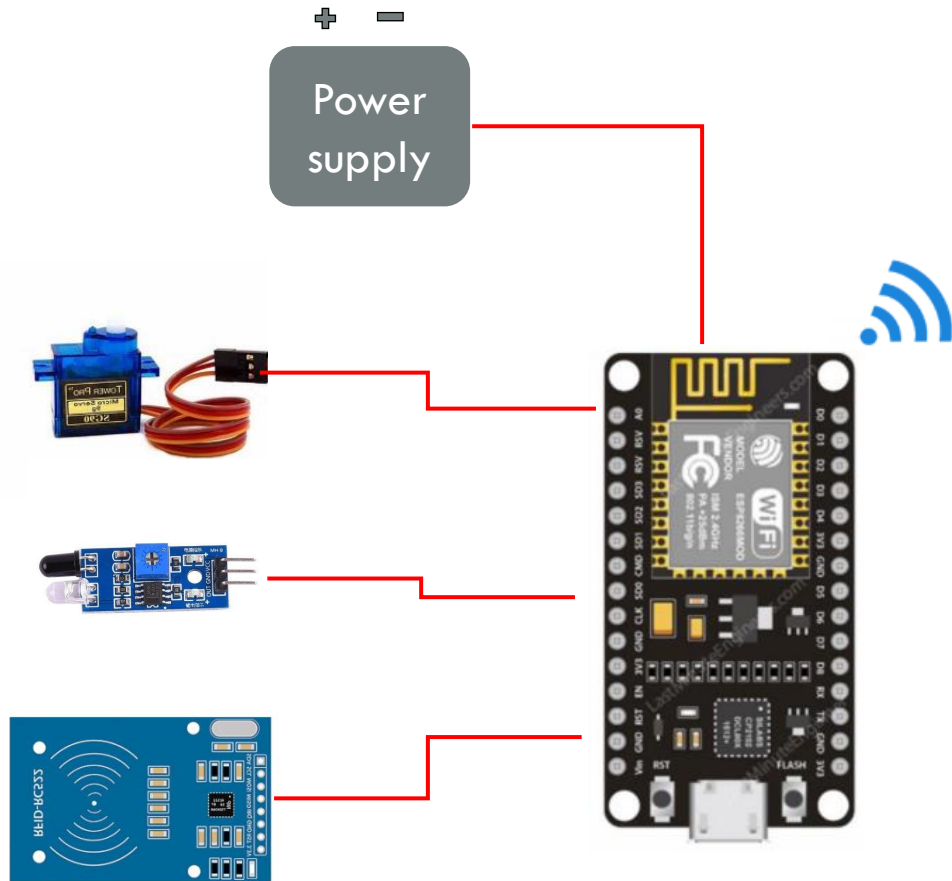
เพิ่มความ
สะดวกในการ
หาที่จอดรถ

ขอบเขตของโครงการ

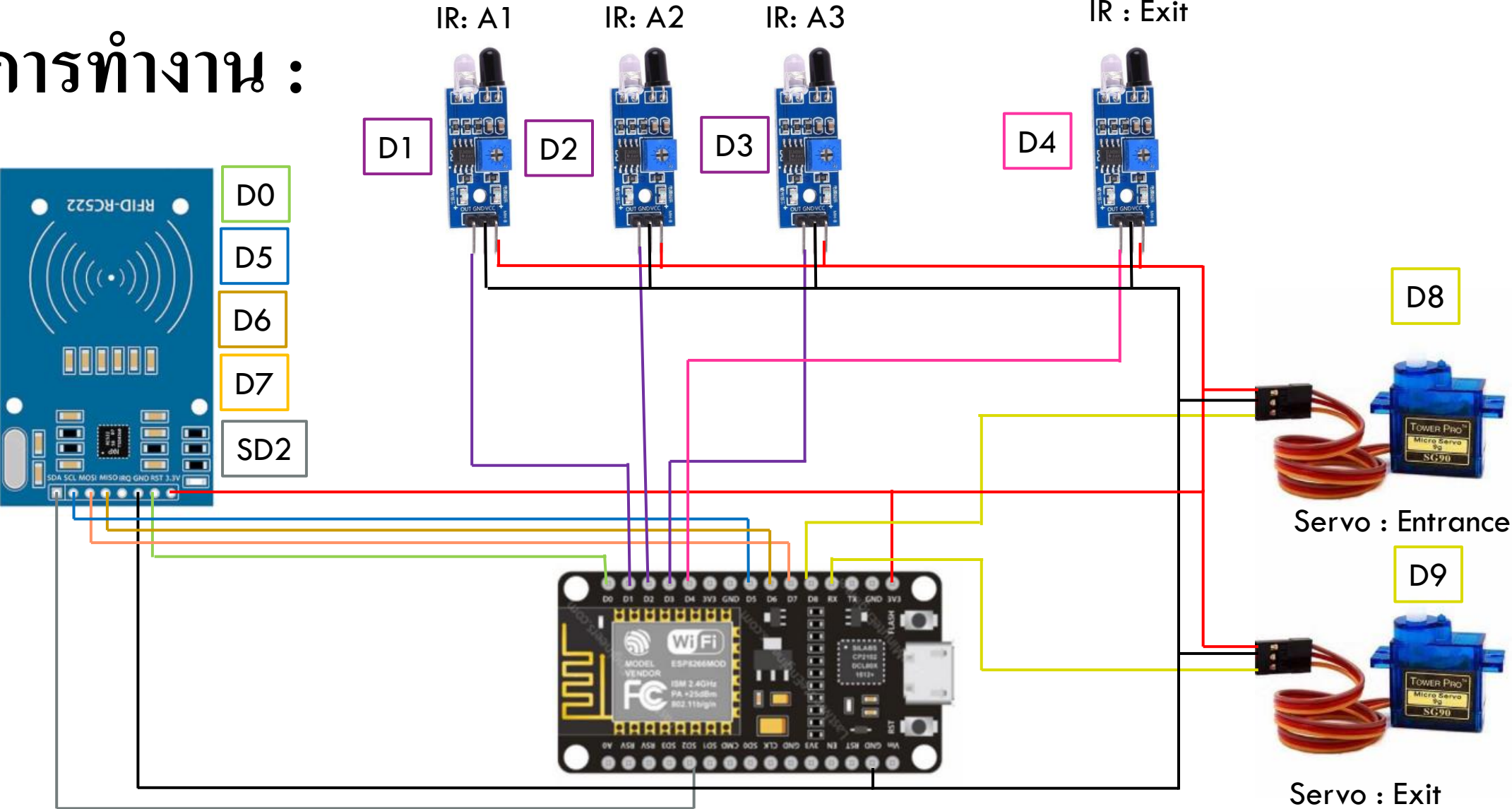
สามารถแสดงสถานะที่
จอรถได้ **3** คันพร้อม
กัน

มีการเสกนบัตรเข้าและ
เปิดประตูทางออกใน
หน้าเว็บ

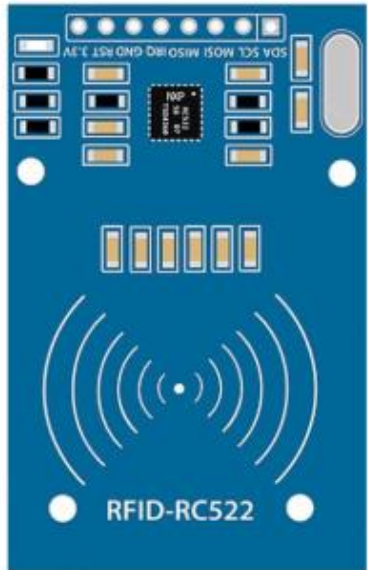
SYSTEM DIAGRAM



การทำงาน :



RFID RC522



สเปกการใช้งานของ RFID RC522 :

- ประเภทของการสื่อสาร: SPI (Serial Peripheral Interface) / UART / I2C
- ความถี่ที่ใช้งาน: 13.56 MHz
- ช่วงการทำงาน: ประมาณ 0-5 ซม. ขึ้นอยู่กับขนาดของแท็ก RFID และแรงดันไฟฟ้าที่ใช้งาน
- รองรับการอ่าน/เขียนแท็ก **RFID**: รองรับแท็กชนิดต่างๆ เช่น MIFARE Classic, MIFARE Ultralight
- แรงดันไฟฟ้าในการทำงาน: 3.3V หรือ 5V
- กระแสไฟฟ้าในการทำงาน: ประมาณ 13-26 mA เมื่อทำงานเต็มที่
- อุณหภูมิที่รองรับ: 0°C ถึง 70°C
- ขนาดโมดูล: ประมาณ 40 มม. x 60 มม.

รายละเอียดของขาต่อในโมดูล RFID RC522:

1.SDA (Serial Data):

1. ใช้สำหรับการสื่อสารผ่าน SPI หรือ I2C.
2. ใน SPI, เป็นขาที่ใช้เลือกบัส (Chip Select, CS) สำหรับโมดูล RFID RC522.

2.SCK (Serial Clock):

1. ขาสัญญาณนาฬิกาสำหรับ SPI (Serial Peripheral Interface).
2. ส่งสัญญาณนาฬิกาเพื่อควบคุมการถ่ายโอนข้อมูล.

3.MOSI (Master Out Slave In):

1. ขานี้ใช้สำหรับการส่งข้อมูลจากไมโครคอนโทรลเลอร์ (Master) ไปยังโมดูล RC522 (Slave) ผ่าน SPI.
2. ส่งข้อมูลที่ต้องการให้ RC522 อ่าน.

4.MISO (Master In Slave Out):

1. ขานี้ใช้สำหรับการรับข้อมูลจากโมดูล RC522 กลับไปยังไมโครคอนโทรลเลอร์.
2. ใช้ในกรณีที่คุณต้องการดึงข้อมูลจากบัตร RFID ที่อ่านได้.

5.IRQ (Interrupt Request):

1. ใช้สำหรับส่งสัญญาณอินเตอร์รัプトไปยังไมโครคอนโทรลเลอร์.
2. เมื่อโมดูล RC522 มีการทำงานบางอย่างเสร็จสมบูรณ์หรือเกิดเหตุการณ์บางอย่าง, ขานี้จะกระตุ้นการทำงานของไมโครคอนโทรลเลอร์.

6.RST (Reset):

1. ขานี้ใช้สำหรับการรีเซ็ตโมดูล RC522.
2. เมื่อส่งสัญญาณ LOW ไปที่ขานี้, โมดูลจะถูกรีเซ็ต.

7.VCC (Power Supply):

1. จ่ายไฟให้กับโมดูล RC522.
2. แรงดันไฟฟ้าที่รองรับคือ 3.3V ถึง 5V ขึ้นอยู่กับไมโครคอนโทรลเลอร์ที่ใช้.

8.GND (Ground):

1. ขาที่ใช้เชื่อมต่อกับพื้น (Ground) เพื่อให้ระบบทำงานได้อย่างถูกต้อง.

SERVO SG90



สเปคการใช้งานของ Servo sg90 :

- สามารถหมุนได้ 90° - 180° (ขึ้นกับรุ่น)
- แรงดัน 5V
- ควบคุมการหมุนด้วย PWM



IR SENSOR



สเปกการใช้งาน :

- แรงดันไฟฟ้าที่ใช้งาน : 3V ถึง 5V
- ระยะการตรวจจับ : ประมาณ 2-80 เซนติเมตร ขึ้นอยู่กับชนิดของเซ็นเซอร์
- มุมการตรวจจับ : ประมาณ 35° - 60° (สำหรับ IR ที่ตรวจจับในระยะใกล้)
- กระแสไฟฟ้าที่ใช้ : ประมาณ 10-30mA
- ขนาด : ขึ้นอยู่กับรุ่นของเซ็นเซอร์ แต่จะมีขนาดค่อนข้างเล็กและง่ายต่อการติดตั้ง

```
//กำหนด libraly การใช้/การใช้งานของ งาน
```

```
#include <ESP8266WiFi.h>
```

```
#include <FirebaseESP8266.h>
```

```
#include <SPI.h>
```

```
#include <RFID.h>
```

```
#include <Servo.h>
```

```
// Define RFID pins
```

```
#define SS_PIN 9
```

```
#define RST_PIN 16
```

```
RFID rfid(SS_PIN, RST_PIN);
```

```
// WiFi
```

```
char ssid[] = "vivo Y21";
```

```
char password[] = "12345678";
```

```
// Firebase
```

```
#define FIREBASE_HOST "realtime-parking-bf00c-default-rtdb.firebaseio.com"
```

```
#define FIREBASE_AUTH "eLLcT97PNTlvgMmAOUyObSqXvmjtrYOCJ7q6Ib0I"
```

```
// Firebase objects
```

```
FirebaseData firebaseData;
```

```
FirebaseConfig firebaseConfig;
```

```
FirebaseAuth firebaseAuth;
```

```
Servo Opan, Exit; // entry and exit
```

```
int sensorValues[4]; // กำหนดตัวแปร ir 4 ตัว
```

```
int serNum[5] = {0}; // กำหนดตัวแปรรหัส id
```

```
#define EXIT_PATH "/Control/Exit"
```

การกำหนดตัวแปรและการประกาศใช้ libraries

•<ESP8266WiFi.h>: ใช้สำหรับเชื่อมต่อ WiFi บนบอร์ด ESP8266

•<FirebaseESP8266.h>: ใช้สำหรับการเชื่อมต่อกับ Firebase Realtime Database

•<SPI.h> และ <RFID.h>: ใช้สำหรับสื่อสารกับโมดูล RFID

•<Servo.h>: ใช้ควบคุม Servo Motor สำหรับเปิด-ปิดไม้กั้น

RFID โมดูล RFID ถูกสร้างขึ้นในตัวแปร rfid

Servo Motor สำหรับควบคุมไม้กั้นทางเข้าและทางออก

IR Sensors ตัวแปร sensorValues[4] ใช้สำหรับเก็บค่าจากเซ็นเซอร์ IR 4 ตัวที่

ตรวจสอบสภาพช่องจอดรถ

serNum[5]: เก็บหมายเลข ID จากบัตร RFID (5 ไบต์)

EXIT_PATH: นิยาม path บน Firebase ที่ใช้ควบคุมการทำงานของไม้กั้นทางออก (เป็น "Control/Exit")

โมดูล RFID อ่าน ID ของบัตรที่ใช้

ส่งข้อมูล ID นี้ไปยัง Firebase Realtime Database

ใช้ Firebase ควบคุมการทำงาน เช่น เปิด-ปิด Servo สำหรับไม้กั้น

เซ็นเซอร์ IR ใช้ตรวจจับสถานะของพื้นที่จอดรถ

```
// Function to update Firebase
void updateFirebase(String path, String value) {
  if (Firebase.setString(firebaseData, path, value)) {
    Serial.println("อัปเดต Firebase สำเร็จ");
  } else {
    Serial.print("Firebase Error: ");
    Serial.println(firebaseData.errorReason());
  }
}

// Function to handle exit gate control from Firebase
void handleExitControl() {
  // พยายามดึงคำสั่งจาก Firebase
  if (Firebase.getString(firebaseData, "/Exit")) {
    String exitCommand = firebaseData.stringData();
    if (exitCommand == "OPEN") {
      Serial.println("เปิดประตูทางออก");
      Exit.write(180); // เปิดประตู
      delay(5000);
      Exit.write(0); // ปิดประตู
      updateFirebase("/Exit", "CLOSED"); // รีเซ็ตคำสั่ง
    }
  } else {
    // หากเส้นทางนี้ไม่มีให้สร้างด้วยค่าเริ่มต้น
    Serial.println("เส้นทาง Exit ไม่มี สร้างขึ้นด้วยค่าเริ่มต้น 'CLOSED'");
    Firebase.setString(firebaseData, "/Exit", "CLOSED");
  }
}
```

Function ต่างๆที่ใช้

ฟังก์ชัน **updateFirebase**:

ฟังก์ชันนี้ใช้สำหรับอัปเดตข้อมูลใน **Firestore**:

พารามิเตอร์

path (String): เส้นทางใน **Firestore** ที่ต้องการอัปเดต

value (String): ค่าที่ต้องการเขียนลงในเส้นทางนั้น

ใช้คำสั่ง **Firebase.setString()** เพื่อเขียนค่า **value** ไปยัง **path**

หากสำเร็จ พิมพ์ข้อความ "อัปเดต **Firestore** สำเร็จ"

หากล้มเหลว แสดงข้อผิดพลาดผ่าน **Serial Monitor**

ฟังก์ชัน **handleExitControl**:

ฟังก์ชันนี้จัดการการควบคุมประตูทางออกตามคำสั่งใน **Firestore**:

ดึงคำสั่งจาก **Firestore**:

ใช้ **Firebase.getString()** เพื่อดึงค่าจากเส้นทาง **"/Exit"**

หากดึงค่าได้สำเร็จ อ่านคำสั่งและตรวจสอบค่า:

หากเป็น **"OPEN"**

พิมพ์ข้อความ "เปิดประตูทางออก"

สั่งให้ **Servo (Exit)** หมุนไปยังตำแหน่ง **180 องศา** เพื่อเปิดประตู

รอ **5 วินาที** แล้วสั่ง **Servo** กลับไปยังตำแหน่ง **0 องศา** เพื่อปิดประตู

อัปเดต **Firestore** ให้รีเซ็ตคำสั่งกลับเป็น **"CLOSED"** ด้วยฟังก์ชัน **updateFirebase**

หากไม่สามารถดึงค่าได้:

พิมพ์ข้อความ "เส้นทาง **Exit** ไม่มี สร้างขึ้นด้วยค่าเริ่มต้น **'CLOSED'**"

สร้างเส้นทาง **"/Exit"** พร้อมค่าเริ่มต้น **"CLOSED"**

```

void handleCardRead() {
    bool isNewCard = false;

    // Check if the card is new
    for (int i = 0; i < 5; i++) {
        if (rfid.serNum[i] != serNum[i]) {
            isNewCard = true;
            break;
        }
    }

    if (isNewCard) {
        memcpy(serNum, rfid.serNum, 5);

        // Convert card ID to string
        String cardID = "";
        for (int i = 0; i < 5; i++) {
            cardID += String(rfid.serNum[i], HEX);
            if (i < 4) cardID += ":";
        }

        Serial.println("ตรวจพบการ์ดใหม่: " + cardID);
        updateFirebase("/CardID", cardID);
    }
}

```

Function ในการอ่านตรวจสอบบัตรใหม่

ตรวจสอบหมายเลขบัตร (rfid.serNum) ว่าต่างจากบัตรเดิม (serNum) หรือไม่
หากไม่ตรงกัน ให้ตั้งค่า isNewCard = true

แปลงบัตรใหม่เป็นสตริง

ใช้สร้าง cardID โดยแปลงตัวเลขแต่ละไบต์ในหมายเลขบัตรเป็นเลขฐาน 16 (HEX) และคั่นด้วยเครื่องหมาย :

```

if (Firebase.getJSON(firebaseData, "/Users")) {
    FirebaseJson users = firebaseData.jsonObject();
    String userKey, userData;
    bool cardFound = false;

    int userCount = users.iteratorBegin();
    int valueType;
    for (int i = 0; i < userCount; i++) {
        users.iteratorGet(i, valueType, userKey, userData);

        // ดึงข้อมูล cardID ของผู้ใช้
        FirebaseJson userJson;
        userJson.setJsonData(userData);

        FirebaseJsonData jsonData;
        userJson.get(jsonData, "cardID");

        if (jsonData.stringValue == cardID) {
            cardFound = true;
            break;
        }
    }
    users.iteratorEnd();
    if (cardFound) {
        Serial.println("บัตรตรงกับข้อมูลในระบบ, เปิดประตู!");
        // Open the entry gate
        Opan.write(180);
        delay(5000);
        Opan.write(0);
    } else {
        Serial.println("บัตรไม่ตรงกับข้อมูลในระบบ");
    }
} else {
    Serial.print("เกิดข้อผิดพลาดในการดึงข้อมูล Firebase: ");
    Serial.println(firebaseData.errorReason());
}

```

ตรวจสอบ Firebase

ดึงข้อมูล JSON จากเส้นทาง /Users

ใช้ FirebaseJson เพื่อจัดการข้อมูลและวนลูป

ตรวจสอบบัตรแต่ละใบ

เปิด-ปิดประตู

หากพบหมายเลขบัตรในระบบ:

ใช้ Servo (Opan) เพื่อเปิดประตู (หมุนไปที่ 180 องศา)

หลังจาก 5 วินาที ปิดประตูโดยหมุนกลับไปที่ 0 องศา

Setup

```
void setup() {  
  Serial.begin(115200);  
  
  // Initialize SPI and RFID  
  SPI.begin();  
  rfid.init();  
  
  // Initialize pins  
  pinMode(D1, INPUT);  
  pinMode(D2, INPUT);  
  pinMode(D3, INPUT);  
  pinMode(D4, INPUT);  
  
  Opan.attach(15); // D8  
  Exit.attach(3);  // D9  
  
  Opan.write(0);  
  Exit.write(0);  
  
  // Connect to WiFi  
  WiFi.begin(ssid, password);  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
  }  
  Serial.println("\nWiFi connected!");  
  Serial.println(WiFi.localIP());  
  
  // Firebase setup  
  firebaseConfig.host = FIREBASE_HOST;  
  firebaseConfig.signer.tokens.legacy_token = FIREBASE_AUTH;  
  Firebase.begin(&firebaseConfig, &firebaseAuth);  
  
  // Check Firebase connection  
  if (Firebase.ready()) {  
    Serial.println("Firebase connected!");  
  } else {  
    Serial.printf("Firebase initialization failed: %s\n", firebaseData.errorReason().c_str());  
  }  
  
  Serial.println("ระบบพร้อมใช้งาน");  
  
  Firebase.setString(firebaseData, "/Exit", "CLOSED");
```

เริ่มต้นโมดูล SPI และ RFID

เริ่มต้น SPI เพื่อสื่อสารกับโมดูล RFID

เรียก `rfid.init()` เพื่อกำหนดค่าเริ่มต้นสำหรับการทำงานของโมดูล RFID

กำหนดพินของเซ็นเซอร์ IR

ตั้งพิน D1, D2, D3, และ D4 เป็นโหมด INPUT เพื่อรับค่าจากเซ็นเซอร์ IR ที่ใช้ตรวจสอบพื้นที่จอดรถ

ตั้งค่า Servo Motor

เชื่อมต่อ Servo Motor สองตัว:

Opan: ใช้สำหรับเปิด-ปิดประตูทางเข้า (เชื่อมกับ D8)

Exit: ใช้สำหรับเปิด-ปิดประตูทางออก (เชื่อมกับ D9)

กำหนดตำแหน่งเริ่มต้นของ Servo ให้หมุนไปที่ 0 องศา (ปิด)

เชื่อมต่อ WiFi

ใช้ `WiFi.begin()` เชื่อมต่อกับ WiFi โดยระบุ ssid และ password

เช็คสถานะการเชื่อมต่อด้วย `WiFi.status()`

หากยังไม่เชื่อมต่อ (สถานะไม่ใช่ WL_CONNECTED): รอ 500ms และพิมพ์จุด (.) แสดงความ

คืบหน้า

เมื่อเชื่อมต่อสำเร็จ: แสดงข้อความ "WiFi connected!" พร้อมแสดง IP Address ของอุปกรณ์

ตั้งค่า Firebase

ระบุค่า Firebase Host และ Legacy Token ใน `firebaseConfig` และเริ่มต้น Firebase

ด้วย `Firebase.begin()`

ตรวจสอบสถานะการเชื่อมต่อ:

หากเชื่อมต่อสำเร็จ: แสดงข้อความ "Firebase connected!"

หากล้มเหลว: แสดงข้อความข้อผิดพลาด (`firebaseData.errorReason()`)

ตั้งค่าเริ่มต้นใน Firebase

เขียนค่าเริ่มต้น "CLOSED" ไปยังเส้นทาง `/Exit` ใน Firebase เพื่อกำหนดสถานะเริ่มต้นของประตูทางออก

```

void loop() {
  // Check RFID card
  if (rfid.isCard() && rfid.readCardSerial()) {
    handleCardRead();
  }

  rfid.halt();

  // Update sensor statuses to Firebase
  sensorValues[0] = digitalRead(D1);
  sensorValues[1] = digitalRead(D2);
  sensorValues[2] = digitalRead(D3);
  sensorValues[3] = digitalRead(D4);

  updateFirebase("/Sensors/Sensor1", sensorValues[0] == LOW ? "FULL" : "EN");
  updateFirebase("/Sensors/Sensor2", sensorValues[1] == LOW ? "FULL" : "EN");
  updateFirebase("/Sensors/Sensor3", sensorValues[2] == LOW ? "FULL" : "EN");

  updateFirebase("/Sensors/SensorD4", digitalRead(D4) == LOW ? "OBJECT_PRESENT" : "NO_OBJECT");

  // Handle exit control (from Firebase)
  handleExitControl(); // Added the semicolon here

  delay(1000); // Prevent frequent Firebase writes
}

```

Loop

ตรวจสอบบัตร RFID

rfid.isCard(): ตรวจสอบว่ามีบัตร RFID อยู่ในระยะที่สามารถอ่านได้หรือไม่

rfid.readCardSerial(): อ่านหมายเลขบัตร (serial number) หากตรวจจับบัตรสำเร็จ
หากมีบัตร

เรียกฟังก์ชัน **handleCardRead()** เพื่อจัดการการตรวจสอบบัตร

rfid.halt(): หยุดการสแกนบัตรชั่วคราวหลังจากอ่านข้อมูลเสร็จ

อัปเดตสถานะเซ็นเซอร์ไปยัง Firebase

อ่านสถานะของเซ็นเซอร์ IR ทั้ง 4 ตัว (D1 ถึง D4) และบันทึกค่าในอาร์เรย์ **sensorValues**
สถานะที่อ่านได้มี 2 แบบ:

LOW: มีวัตถุอยู่ (พื้นที่เต็มหรือมีวัตถุ)

HIGH: ไม่มีวัตถุอยู่ (พื้นที่ว่าง)

เขียนสถานะเซ็นเซอร์ไปยัง Firebase:

FULL: พื้นที่เต็ม

EN: พื้นที่ว่าง

เซ็นเซอร์ **D4** ถูกกำหนดให้แสดงสถานะเฉพาะวัตถุที่อยู่หรือไม่
(เช่น "OBJECT_PRESENT" หรือ "NO_OBJECT")

ควบคุมประตูทางออกตามคำสั่ง Firebase

เรียกฟังก์ชัน **handleExitControl()** เพื่อตรวจสอบคำสั่งการเปิด-ปิดประตูทางออกจาก Firebase
หากคำสั่งเป็น "OPEN":

เปิดประตูทางออก (Servo Exit หมุนไปที่ 180 องศา)

ปิดประตูหลังจากรอ 5 วินาที

รีเซ็ตคำสั่งใน Firebase เป็น "CLOSED"

```

import React, { useState, useEffect } from 'react';
import { initializeApp } from 'firebase/app';
import { getDatabase, ref, onValue, set, get } from 'firebase/database';
import { getStorage } from 'firebase/storage';
import './App.css';
import car1Image from './img/car1.png';

function App() {
  const firebaseConfig = {
    apiKey: 'AIzaSyBQjpw-E5-K6y60R3gD97moqdMaIMik',
    authDomain: 'realtime-parking-bf00c.firebaseio.com',
    databaseURL: 'https://realtime-parking-bf00c-default-rtdb.firebaseio.com',
    projectId: 'realtime-parking-bf00c',
    storageBucket: 'realtime-parking-bf00c.firebaseio.com',
    messagingSenderId: '310377636017',
    appId: '1:310377636017:web:8091cfbad496b2e4f08290',
    measurementId: 'G-7260PMWZ8L',
  };

  const app = initializeApp(firebaseConfig);
  const database = getDatabase(app);
  const storage = getStorage(app);

  const [sensorStates, setSensorStates] = useState([
    D1: false,
    D2: false,
    D3: false,
    D4: false,
  ]);

  // Fetch sensor states
  useEffect(() => {
    const sensorRef = ref(database, '/Sensors');
    onValue(sensorRef, (snapshot) => {
      const data = snapshot.val();
      if (data) {
        setSensorStates({
          D1: data.Sensor1 === 'FULL',
          D2: data.Sensor2 === 'FULL',
          D3: data.Sensor3 === 'FULL',
          D4: data.SensorD4 === 'OBJECT_PRESENT',
        });
      }
    });
  }, [database]);

```

การตั้งค่าการเชื่อมต่อ **Firestore**

firebaseConfig เก็บค่าคอนฟิกที่จำเป็นสำหรับการเชื่อมต่อ **Firestore Project**

initializeApp(firebaseConfig) เริ่มต้น **Firestore App** ด้วยคอนฟิกที่กำหนด

การกำหนดฐานข้อมูล **Firestore**

getDatabase(app) เรียกใช้งาน **Realtime Database** ของ **Firestore**

getStorage(app) เรียกใช้งาน **Firestore Storage** (ยังไม่ใช้ในโค้ดนี้ แต่เตรียมไว้สำหรับอนาคต)

การจัดการสถานะเซ็นเซอร์

ใช้ **useState** เพื่อเก็บสถานะของเซ็นเซอร์ 4 ตัว (D1-D4)

ค่าเริ่มต้นเป็น **false** หมายถึงยังไม่มีวัตถุอยู่ในพื้นที่

การดึงข้อมูลเซ็นเซอร์จาก **Firestore**

useEffect:

ดึงข้อมูลเซ็นเซอร์จาก **Firestore** เส้นทาง **/Sensors**

ใช้ **onValue** เพื่อฟังการเปลี่ยนแปลงของข้อมูลในแบบเรียลไทม์

การกำหนดค่า **sensorStates:**

ตรวจสอบสถานะของเซ็นเซอร์แต่ละตัวตามค่าที่ได้รับจาก **Firestore:**

"FULL": เซ็นเซอร์ตรวจจับวัตถุ (พื้นที่เต็ม)

"OBJECT_PRESENT": เซ็นเซอร์ D4 ตรวจจับวัตถุอยู่ในพื้นที่

```

const [showPopup, setShowPopup] = useState(false);
const [userName, setUserName] = useState('');

useEffect(() => {
  const cardRef = ref(database, '/CardID');

  onValue(cardRef, async (snapshot) => {
    const cardID = snapshot.val();
    console.log("555");
    if (cardID) {
      try {
        // ดึงข้อมูลทั้งหมดจาก '/Users'
        const usersRef = ref(database, '/Users');
        const usersSnapshot = await get(usersRef);

        if (usersSnapshot.exists()) {
          const usersData = usersSnapshot.val();
          console.log("666");
          // วนลูปเช็ค cardID ของแต่ละผู้ใช้
          Object.values(usersData).forEach((userData) => {
            if (userData.cardID === cardID) {
              // แสดงpopup
              console.log("333");
              setUserName(userData.firstname);
              setShowPopup(true); // Trigger popup display
            }
          });
        }
      } catch (error) {
        console.error('เกิดข้อผิดพลาดในการดึงข้อมูลผู้ใช้:', error);
      }
    }
  });
}, [database]);

```

Webside

การประกาศตัวแปร **state**

showPopup: ตัวแปรที่ใช้จัดการการแสดงผล Popup ว่าจะให้แสดงหรือไม่

userName: ตัวแปรที่เก็บชื่อผู้ใช้ ซึ่งจะใช้ในการแสดงใน Popup

การเชื่อมต่อกับ **Firebase** ใน **useEffect**

การทำงานของโค้ด

ใช้ **useEffect** เพื่อติดตามการเปลี่ยนแปลงของ **CardID** ใน **Firebase Realtime Database (/CardID)**:

onValue(cardRef, async (snapshot) => {...}):

เมื่อ **CardID** ใน **Firebase** มีการเปลี่ยนแปลง (อาจมาจากการสแกน **RFID** หรืออัปเดต), ระบบจะดึงข้อมูล **cardID** จาก **Firebase**.

เมื่อ **cardID** มีค่า:

get(usersRef): ดึงข้อมูลจาก **Firebase** เส้นทาง **/Users** ทั้งหมด

Object.values(usersData).forEach(...): วนลูปเพื่อหาข้อมูลผู้ใช้ใน **usersData**

ตรวจสอบว่า **cardID** ตรงกับของผู้ใช้หรือไม่:

ถ้าตรง:

setUserName(userData.firstname): เชื่อมชื่อผู้ใช้ใน **state userName**

setShowPopup(true): แสดง **Popup**

การจัดการข้อผิดพลาด:

ถ้ามีข้อผิดพลาดในการดึงข้อมูลจาก **Firebase** เช่น การเชื่อมต่อล้มเหลว หรือข้อมูลไม่ถูกต้อง ระบบจะแสดงข้อผิดพลาดในคอนโซล

```

const handleExit = () => {
  const exitRef = ref(database, '/Exit');
  set(exitRef, 'OPEN');
};

return (
  <div>
    <header>
      <nav>
        <div className="logo">
          <h1>Realtime Parking</h1>
        </div>
      </nav>
    </header>
    {showPopup && (
      <div className="popmodel">
        <div className="popup">
          <p>{userName} ENTRY</p>
          <button onClick={() => setShowPopup(false)}>Close</button>
        </div>
      </div>
    )}

    <section>
      <div className="main-content">
        <div id="park">
          <div className="barall">
            <div className="bar"></div>
            <div className="bar"></div>
            <div className="bar"></div>
            <div className="bar"></div>
          </div>
        </div>
      </section>
    </div>
  );

```

```

{({sensorStates.D1 && sensorStates.D2 && sensorStates.D3}) ? (
  <div className="full-message">
    <p>This parking lot is full</p>
  </div>
  <div className="cary1">
    <img className="car1-display" src={car1Image} alt="Car 1" />
  </div>
  <div className="cary2">
    <img className="car2-display" src={car1Image} alt="Car 2" />
  </div>
  <div className="cary3">
    <img className="car3-display" src={car1Image} alt="Car 3" />
  </div>
</div>
): (
  <div className="car1">
    <img
      className="car1-display"
      src={car1Image}
      alt="Car 1"
      style={{ display: sensorStates['D1'] ? 'block' : 'none' }}
    />
    <h2 className="log1" style={{ display: sensorStates['D1'] ? 'none' : 'block' }}>A1</h2>
  </div>
  <div className="car2">
    <img
      className="car2-display"
      src={car1Image}
      alt="Car 2"
      style={{ display: sensorStates['D2'] ? 'block' : 'none' }}
    />
    <h2 className="log2" style={{ display: sensorStates['D2'] ? 'none' : 'block' }}>A2</h2>
  </div>
  <div className="car3">
    <img
      className="car3-display"
      src={car1Image}
      alt="Car 3"
      style={{ display: sensorStates['D3'] ? 'block' : 'none' }}
    />
    <h2 className="log3" style={{ display: sensorStates['D3'] ? 'none' : 'block' }}>A3</h2>
  </div>
</div>
);

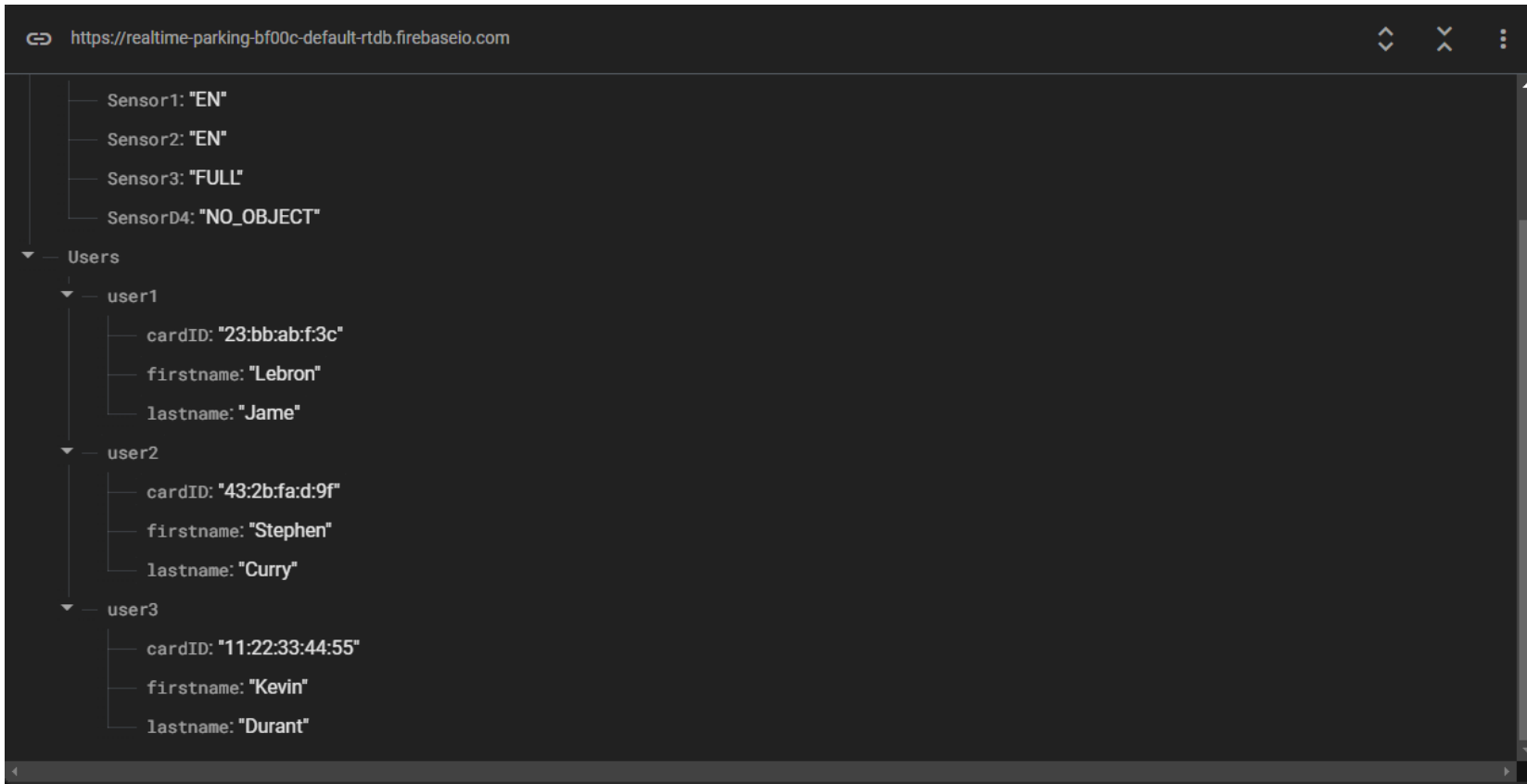
```

```

    <div className="exit">
      <div className="exit-box">
        <button
          onClick={handleExit}
          disabled={!sensorStates.D4}
          className={`btn ${sensorStates.D4 ? 'enabled' : 'disabled'}}`>
          EXIT
        </button>
      </div>
    </div>
  </div>
</div>
</section>
</div>
];
}

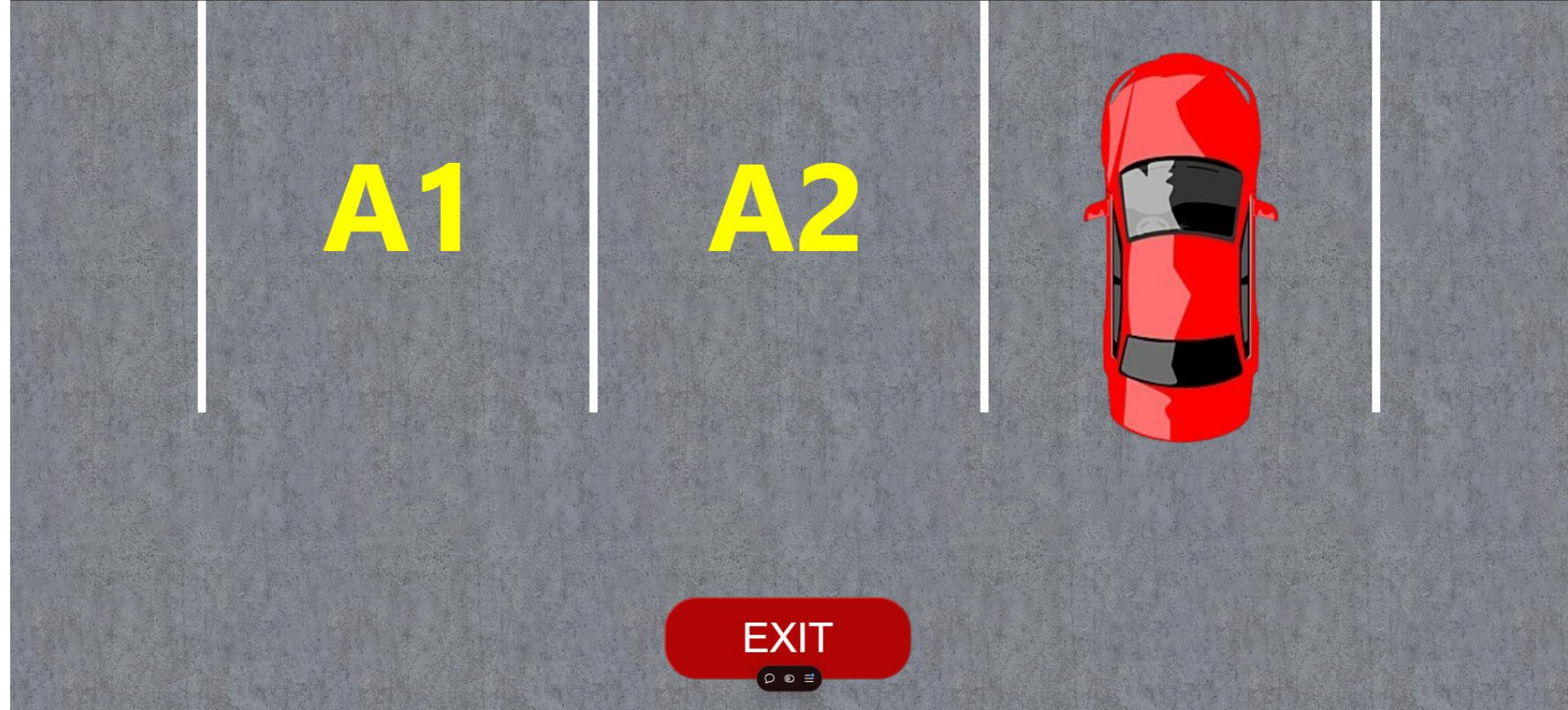
```

```
export default App;
```



Realtime firebase

Realtime Parking



ตัวอย่างหน้าเว็บเบื้องต้น <https://realtimeparking.vercel.app/>